



Basi di Dati
Progetto A.A. 2025/2026

Sistema Informativo di una Pizzeria

0292126
Dario Sella

Indice

1. Descrizione del Minimondo.....	2
2. Analisi dei Requisiti.....	3
3. Progettazione concettuale.....	6
4. Progettazione logica.....	10
5. Progettazione fisica.....	19

1. Descrizione del Minimondo

- | | |
|----|---|
| 1 | Si vuole realizzare un sistema di gestione di una pizzeria, operante 7 giorni su 7 sia a |
| 2 | pranzo che a cena. La pizzeria opera in loco, tramite asporto o con consegne. |
| 3 | Quando un cliente entra in pizzeria, se vuole consumare sul posto, gli viene assegnato un |
| 4 | tavolo libero. I camerieri prendono le comande andando al tavolo ed aggiungendo le |
| 5 | ordinazioni all'elenco dei prodotti da preparare. I prodotti, appartenenti ad un menu fisso |
| 6 | gestito dal proprietario del locale, sono pizze o bevande. I pizzaioli possono consultare il |
| 7 | sistema per sapere quali pizze devono essere preparate. Il barman può fare lo stesso per |
| 8 | preparare le bevande. |
| 9 | Quando una pizza o una bevanda (di una certa comanda) è pronta, il pizzaiolo o il barman |
| 10 | la indicano come preparata. I camerieri possono consultare dai loro terminali quali prodotti |
| 11 | sono pronti e a quali tavoli vanno consegnati. |
| 12 | In caso di consegne, l'ordine viene acquisito tramite telefono. In questo caso, il proprietario |
| 13 | registra l'indirizzo di consegna e il nome del cliente, oltre ad un recapito telefonico. |
| 14 | In caso di ordine da asporto o in consegna, il manager ritira sia pizze che bevande dalla |
| 15 | cucina, per consegnarle al cliente o al fattorino. |
| 16 | Al termine del pasto o al ritiro dell'ordine, il gestore segna la comanda come completata e |
| 17 | può stampare lo scontrino non fiscale di riepilogo. Per motivi statistici, ha la possibilità di |
| 18 | visualizzare le entrate giornaliere e/o mensili. |

2. Analisi dei Requisiti

Identificazione dei termini ambigui e correzioni possibili

Linea	Termine	Nuovo termine	Motivo correzione
5	elenco dei prodotti da preparare	comanda	Il termine comanda è una definizione semplice e chiara dello stesso concetto.
6	proprietario	manager	Il cliente ha chiarito che le attività operative spettano al manager, che può coincidere con il proprietario.
10	preparata	pronta	Il nuovo termine evita interpretazioni ambigue legate al processo di preparazione.
11	consegnati	serviti	Il tavolo viene servito dal cameriere, la consegna deve essere relativa all'ordine.
12	consegne	ordini in consegna	Il nuovo termine distingue bene il tipo di ordine dall'azione.
12	telefono	chiamata telefonica	Il cliente ha chiarito che si tratta di una chiamata telefonica.
12	proprietario	manager	Il cliente ha chiarito che le attività operative spettano al manager, che può coincidere con il proprietario.
16	gestore	manager	Il termine gestore è un sinonimo di manager.

Specificazione disambiguata

Si vuole realizzare un sistema di gestione di una pizzeria, operante 7 giorni su 7 sia a pranzo che a cena. La pizzeria opera in loco, tramite asporto o con consegne.

Quando un cliente entra in pizzeria, se vuole consumare sul posto, gli viene assegnato un tavolo libero. I camerieri prendono le comande andando al tavolo ed aggiungendo le ordinazioni alla comanda. I prodotti, appartenenti ad un menu fisso gestito dal manager del locale, sono pizze o bevande. I pizzaioli possono consultare il sistema per sapere quali pizze devono essere preparate. Il barman può fare lo stesso per preparare le bevande.

Quando una pizza o una bevanda (di una certa comanda) è pronta, il pizzaiolo o il barman la indicano come pronta. I camerieri possono consultare dai loro terminali quali prodotti sono pronti e a quali tavoli vanno serviti.

In caso di ordini in consegna, l'ordine viene acquisito tramite chiamata telefonica. In questo caso, il manager registra l'indirizzo di consegna e il nome del cliente, oltre ad un recapito telefonico.

In caso di ordine da asporto o in consegna, il manager ritira sia pizze che bevande dalla cucina, per consegnarle al cliente o al fattorino.

Al termine del pasto o al ritiro dell'ordine, il manager segna la comanda come completata e può

stampare lo scontrino non fiscale di riepilogo. Per motivi statistici, ha la possibilità di visualizzare le entrate giornaliere e/o mensili.

Glossario dei Termini

Termine	Descrizione	Sinonimi	Collegamenti
Tavolo	Tavolo del locale. Se è libero viene assegnato ad un cliente che consuma sul posto. Viene servito dal cameriere.		Cliente, Cameriere, Comanda, Prodotto
Cliente	Persona che consuma al tavolo oppure che effettua un ordine da asporto al banco o un ordine in consegna tramite chiamata telefonica.		Tavolo, Ordine, Manager
Cameriere	Dipendente del locale che prende le comande ai tavoli e serve ai tavoli.		Tavolo, Comanda, Prodotto
Pizzaiolo	Dipendente del locale che prepara le pizze che vengono ordinate.		Prodotto
Barman	Dipendente del locale che prepara le bevande che vengono ordinate.		Prodotto
Prodotto	Prodotto presente sul menu, può essere una pizza o una bevanda.		Menu, Pizzaiolo, Barman, Comanda
Manager	Dipendente o proprietario del locale che gestisce il menu, registra i dati del cliente, consegna l'ordine da asporto al cliente o l'ordine in consegna al fattorino, segna la comanda come completata e stampa lo scontrino. Può visualizzare le entrate giornaliere e/o mensili.		Menu, Cliente, Fattorino, Comanda, Scontrino
Menu	Elenco dei prodotti offerti dal locale. Viene gestito dal		Prodotto, Manager

	Manager.		
Comanda	Elenco dei prodotti da preparare per un Tavolo oppure per un Ordine.		Cameriere, Tavolo, Manager, Prodotto, Ordine
Ordine	Richiesta del cliente che deve essere da asporto o in consegna.		Comanda, Cliente, Manager, Fattorino
Fattorino	Dipendente del locale che consegna gli ordini in consegna al relativo indirizzo.		Ordine, Manager

Raggruppamento dei requisiti in insiemi omogenei

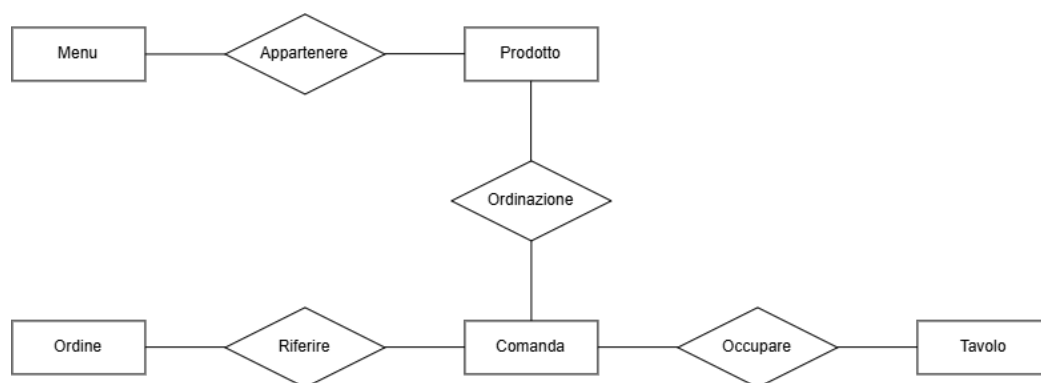
Fraasi relative a Tavolo
Quando un cliente entra in pizzeria, se vuole consumare sul posto, gli viene assegnato un tavolo libero. I camerieri prendono le comande andando al tavolo ed aggiungendo le ordinazioni alla comanda. I camerieri possono consultare dai loro terminali quali prodotti sono pronti e a quali tavoli vanno serviti.
Fraasi relative a Comanda
I camerieri prendono le comande andando al tavolo ed aggiungendo le ordinazioni alla comanda. Quando una pizza o una bevanda (di una certa comanda) è pronta, il pizzaiolo o il barman la indicano come pronta. Al termine del pasto o al ritiro dell'ordine, il manager segna la comanda come completata e può stampare lo scontrino non fiscale di riepilogo.
Fraasi relative a Ordine
In caso di ordini in consegna, l'ordine viene acquisito tramite chiamata telefonica. In questo caso, il manager registra l'indirizzo di consegna e il nome del cliente, oltre ad un recapito telefonico. In caso di ordine da asporto o in consegna, il manager ritira sia pizze che bevande dalla cucina, per consegnarle al cliente o al fattorino.
Fraasi relative a Prodotto
I prodotti, appartenenti ad un menu fisso gestito dal manager del locale, sono pizze o bevande. I pizzaioli possono consultare il sistema per sapere quali pizze devono essere preparate. Il barman può fare lo stesso per preparare le bevande. Quando una pizza o una bevanda (di una certa comanda) è pronta, il pizzaiolo o il barman la indicano come pronta. I camerieri possono consultare dai loro terminali quali prodotti sono pronti e a quali tavoli vanno serviti.
Fraasi relative a Menu
I prodotti, appartenenti ad un menu fisso gestito dal manager del locale, sono pizze o bevande.
Fraasi relative a Manager
I prodotti, appartenenti ad un menu fisso gestito dal manager del locale, sono pizze o bevande. Il manager registra l'indirizzo di consegna e il nome del cliente, oltre ad un recapito telefonico. Il manager ritira sia pizze che bevande dalla cucina, per consegnarle al cliente o al fattorino. Al termine del pasto o al ritiro dell'ordine, il manager segna la comanda come completata e può stampare lo scontrino non fiscale di riepilogo. Per motivi statistici, ha la possibilità di visualizzare le entrate giornaliere e/o mensili.

3. Progettazione concettuale

Costruzione dello schema E-R

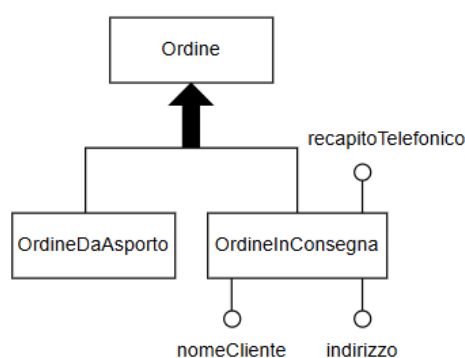
La costruzione dello schema E-R è stata condotta utilizzando la strategia mista.

1. Sono partito individuando i concetti più rilevanti e rappresentandoli tramite uno schema scheletro.



2. Con riferimento allo schema scheletro, ho eseguito il passo iterativo sull'entità Ordine.

Analizzando la specifica relativa si individuano due tipologie di ordine: ordine da asporto e ordine in consegna. Questi concetti sono rappresentabili come entità figlie dell'entità Ordine tramite una generalizzazione totale e disgiunta.

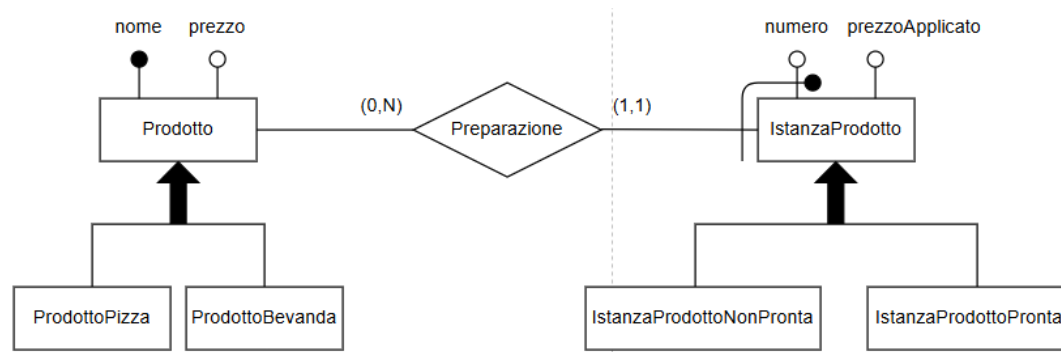


3. Con riferimento allo schema scheletro, ho eseguito il passo iterativo sull'entità Prodotto.

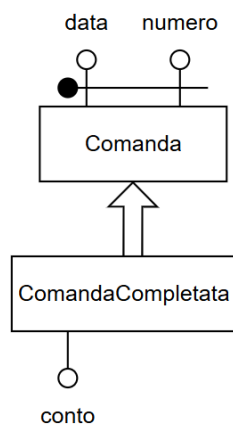
Analizzando la specifica relativa si individuano due tipologie di prodotto: pizza e bevanda.

Questi concetti sono rappresentabili come entità figlie dell'entità Prodotto tramite una generalizzazione totale e disgiunta. Ho ritenuto necessario distinguere il prodotto astratto del menu dal prodotto reale preparato dalla cucina per cui ho utilizzato il pattern Instance-of tramite la relazione Preparazione. L'entità IstanzaProdotto è un'entità debole e utilizza un identificatore esterno, inoltre ho introdotto l'attributo "prezzoApplicato" che rende indipendente la modifica dei prezzi del menu dallo storico. L'istanza del prodotto può essere non pronta oppure pronta, per cui ho utilizzato una generalizzazione totale e disgiunta per

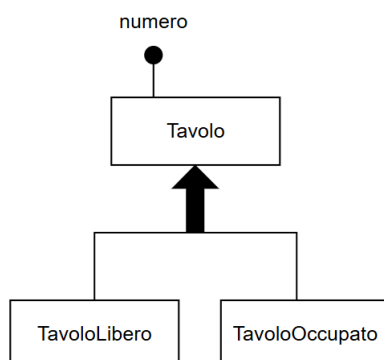
rappresentare i due casi.



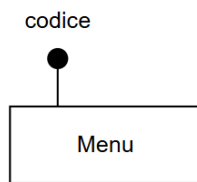
4. Con riferimento allo schema scheletro, ho eseguito il passo iterativo sull'entità Comanda. Ho ritenuto necessario caratterizzare la comanda con data e numero giornaliero che formano un identificatore composto. Analizzando la specifica relativa si individua un'evoluzione nel tempo della comanda che diventa completata, il costrutto che ho ritenuto adatto a rappresentare questo caso è quello della generalizzazione parziale. Infine, ho aggiunto l'attributo "conto" che indica il prezzo totale pagato dal cliente.



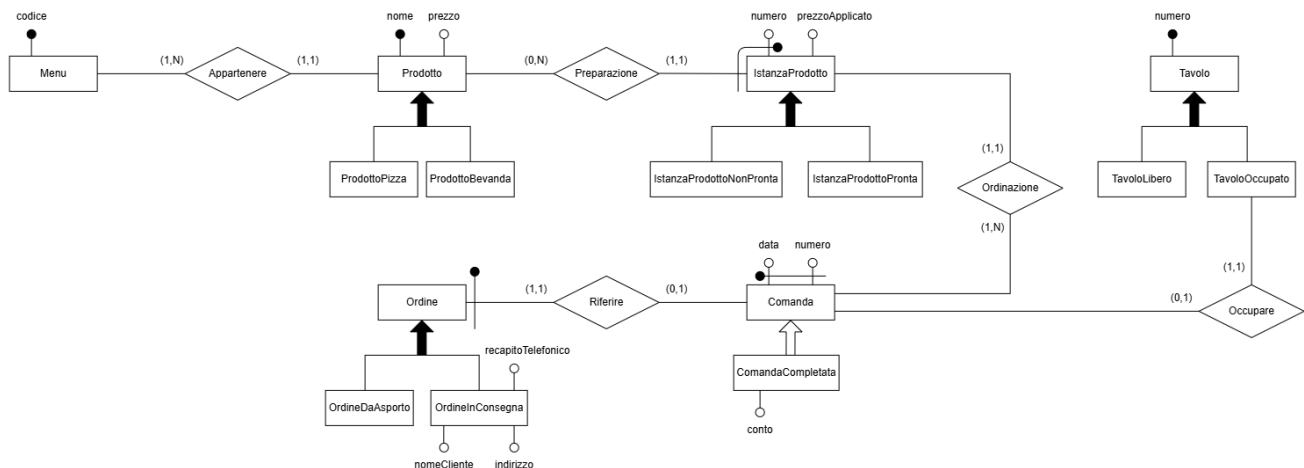
5. Con riferimento allo schema scheletro, ho eseguito il passo iterativo sull'entità Tavolo. Il tavolo viene identificato dal numero. Analizzando la specifica relativa si individua che il tavolo può essere libero oppure occupato, per cui ho rappresentato la situazione con una generalizzazione totale e disgiunta.



6. Con riferimento allo schema scheletro, ho eseguito il passo iterativo sull'entità Menu.



Integrazione finale



Regole aziendali

Regole di vincolo

1. Ogni comanda deve essere associata a un tavolo oppure a un ordine, ma non a entrambi.
2. Il numero delle comande deve partire da zero ogni giorno.
3. Il prezzo di un prodotto deve essere maggiore di zero.
4. Una comanda non deve essere completata se contiene istanze di prodotti non pronte.

Regole di derivazione

1. Il conto di una comanda completata si ottiene sommando il prezzo applicato di tutte le istanze di prodotto appartenenti a quella comanda.
2. L'entrata giornaliera si ottiene sommando i conti delle comande completate in quel giorno.
3. L'entrata mensile si ottiene sommando i conti delle comande completate in quel mese.
4. Il prezzo applicato a un'istanza di prodotto si ottiene copiando il prezzo del prodotto di cui è istanza.

Dizionario dei dati

Entità	Descrizione	Attributi	Identificatori
Comanda	Elenco dei prodotti da preparare per un tavolo oppure per un ordine.	Numero, Data	Numero, Data

ComandaCompleta	Comanda chiusa e pagata dal cliente.	Numero, Data, Conto	Numero, Data
Ordine	Richiesta del cliente che non siede al tavolo.		Comanda(Numero, Data)
OrdineDaAsporto	Richiesta del cliente da asporto.		Comanda(Numero, Data)
OrdineInConsegna	Richiesta del cliente in consegna.	NomeCliente, Indirizzo, RecapitoTelefonico	Comanda(Numero, Data)
Prodotto	Prodotto presente sul menu.	Nome, Prezzo	Nome
ProdottoPizza	Prodotto che è una pizza.	Nome, Prezzo	Nome
ProdottoBevanda	Prodotto che è una bevanda.	Nome, Prezzo	Nome
IstanzaProdotto	Prodotto fisico che verrà preparato in cucina.	Numero, PrezzoApplicato	Prodotto(Nome), Numero
IstanzaProdottoNonPronta	Prodotto fisico non ancora preparato dalla cucina.	Numero, PrezzoApplicato	Prodotto(Nome), Numero
IstanzaProdottoPronta	Prodotto fisico preparato dalla cucina e pronto per essere servito.	Numero, PrezzoApplicato	Prodotto(Nome), Numero
Tavolo	Tavolo del locale.	Numero	Numero
TavoloLibero	Tavolo libero.	Numero	Numero
TavoloOccupato	Tavolo occupato dal cliente.	Numero	Numero
Menu	Elenco dei prodotti offerti dal locale.	Codice	Codice

4. Progettazione logica

Volume dei dati

Ai fini della stima del volume dei dati si assume che le informazioni relative alle comande, agli ordini e alle istanze di prodotto vengano conservate nel database per almeno un anno. La pizzeria dispone di un unico menu fisso, composto da 40 prodotti, di cui 25 pizze e 15 bevande, e di 20 tavoli. Si prevede la gestione di 50 comande al giorno: 30 per il consumo in loco, 12 per l'asporto e 8 per la consegna. Ogni comanda comprende mediamente 4 istanze di prodotto.

Concetto nello schema	Tipo ¹	Volume atteso
Menu	E	1
Prodotto	E	40
ProdottoPizza	E	25
ProdottoBevanda	E	15
Tavolo	E	20
TavoloLibero	E	20
TavoloOccupato	E	20
Comanda	E	18.250
ComandaCompletata	E	18.250
Ordine	E	7.300
OrdineDaAsporto	E	4.380
OrdineInConsegna	E	2.920
IstanzaProdotto	E	73.000
IstanzaProdottoPronta	E	73.000
IstanzaProdottoNonPronta	E	100
Appartenere	R	40
Preparazione	R	73.000
Ordinazione	R	73.000
Riferire	R	7.300
Occupare	R	20

Tavola delle operazioni

Cod.	Descrizione	Frequenza attesa
OP1	Aprire una comanda e associarla a un tavolo	30/giorno
OP2	Aggiungere un'ordinazione a una comanda	200/giorno
OP3	Consultare le pizze da preparare	25/giorno
OP4	Consultare le bevande da preparare	25/giorno
OP5	Segnalare una pizza come pronta	100/giorno
OP6	Segnalare una bevanda come pronta	100/giorno

¹ Indicare con E le entità, con R le relazioni

OP7	Consultare i prodotti pronti e il relativo tavolo	30/giorno
OP8	Completare una comanda e calcolarne il conto	50/giorno
OP9	Generare lo scontrino non fiscale di riepilogo	50/giorno
OP10	Visualizzare le entrate giornaliere	1/giorno
OP11	Visualizzare le entrate mensili	1/mese
OP12	Liberare un tavolo	30/giorno
OP13	Registrare un ordine da asporto	12/giorno
OP14	Registrare un ordine in consegna	8/giorno
OP15	Modificare il prezzo di un prodotto	40/anno

Costo delle operazioni

OP1

Aprire una comanda e associarla a un tavolo.

Frequenza: 30/giorno

Costo totale: 270/giorno

Concetto	Costrutto	Accessi	Tipo
TavoloLibero	E	1	L
TavoloLibero	E	1	S
TavoloOccupato	E	1	S
Comanda	E	1	S
Occupare	E	1	S

OP2

Aggiungere un'ordinazione a una comanda.

Frequenza: 200/giorno

Costo totale: 1.600/giorno

Concetto	Costrutto	Accessi	Tipo
Comanda	E	1	L
Prodotto	E	1	L
IstanzaProdottoNonPronta	E	1	S
Preparazione	R	1	S
Ordinazione	R	1	S

OP3

Consultare le pizze da preparare.

Frequenza: 25/giorno

Costo totale: 300/giorno

Concetto	Costrutto	Accessi	Tipo
IstanzaProdottoNonPronta	E	4	L
Preparazione	R	4	L
ProdottoPizza	E	4	L

OP4

Consultare le bevande da preparare.

Frequenza: 25/giorno

Costo totale: 300/giorno

Concetto	Costrutto	Accessi	Tipo
IstanzaProdottoNonPronta	E	4	L
Preparazione	R	4	L
ProdottoBevanda	E	4	L

OP5

Segnalare una pizza come pronta.

Frequenza: 100/giorno

Costo totale: 700/giorno

Concetto	Costrutto	Accessi	Tipo
IstanzaProdottoNonPronta	E	1	L
Preparazione	R	1	L
ProdottoPizza	E	1	L
IstanzaProdottoNonPronta	E	1	S
IstanzaProdottoPronta	E	1	S

OP6

Segnalare una bevanda come pronta

Frequenza: 100/giorno

Costo totale: 700/giorno

Concetto	Costrutto	Accessi	Tipo
IstanzaProdottoNonPronta	E	1	L
Preparazione	R	1	L
ProdottoBevanda	E	1	L
IstanzaProdottoNonPronta	E	1	S
IstanzaProdottoPronta	E	1	S

OP7

Consultare i prodotti pronti e il relativo tavolo.

Frequenza: 30/giorno

Costo totale: 840/giorno

Concetto	Costrutto	Accessi	Tipo
IstanzaProdottoPronta	E	4	L
Preparazione	R	4	L
Prodotto	E	4	L
Ordinazione	R	4	L
Comanda	E	4	L
Occupare	R	4	L
TavoloOccupato	E	4	L

OP8

Completare una comanda e calcolarne il conto.

Frequenza: 50/giorno

Costo totale: 550/giorno

Concetto	Costrutto	Accessi	Tipo
Comanda	E	1	L
Ordinazione	R	4	L
IstanzaProdotto	E	4	L
ComandaCompletata	E	1	S

OP9

Generare lo scontrino non fiscale.

Frequenza: 50/giorno

Costo totale: 850/giorno

Concetto	Costrutto	Accessi	Tipo
ComandaCompletata	E	1	L
Ordinazione	R	4	L
IstanzaProdotto	E	4	L
Preparazione	R	4	L
Prodotto	E	4	L

OP10

Visualizzare le entrate giornalieri.

Frequenza: 1/giorno

Costo totale: 50/giorno

Concetto	Costrutto	Accessi	Tipo
ComandaCompletata	E	50	L

OP11

Visualizzare le entrate mensili.

Frequenza: 1/mese

Costo totale: 1.521/mese

Concetto	Costrutto	Accessi	Tipo
ComandaCompletata	E	1.521	L

OP12

Liberare un tavolo.

Frequenza: 30/giorno

Costo totale: 240/giorno

Concetto	Costrutto	Accessi	Tipo
ComandaCompletata	E	1	L
Occupare	R	1	L
Occupare	R	1	S
TavoloOccupato	E	1	S
TavoloLibero	E	1	S

OP13

Registrare un ordine da asporto.

Frequenza: 12/giorno

Costo totale: 72/giorno

Concetto	Costrutto	Accessi	Tipo
Comanda	E	1	S
OrdineDaAsporto	E	1	S
Riferire	R	1	S

OP14

Registrare un ordine in consegna.

Frequenza: 8/giorno

Costo totale: 48/giorno

Concetto	Costrutto	Accessi	Tipo
Comanda	E	1	S
OrdineInConsegna	E	1	S
Riferire	R	1	S

OP15

Modificare il prezzo di un prodotto.

Frequenza: 40/anno

Costo totale: 80/anno

Concetto	Costrutto	Accessi	Tipo
Prodotto	E	1	S

Ristrutturazione dello schema E-R**Analisi delle ridondanze**

L'attributo "conto" di ComandaCompletata può essere derivato, tramite l'associazione Ordinazione, sommando i "prezzoApplicato" delle istanze di prodotto che compongono la comanda.

Facendo riferimento alla Tavola dei volumi e assumendo che l'attributo "conto" richieda 4 byte, esso comporta annualmente un'occupazione di memoria aggiuntiva pari a 73.000 byte.

Le operazioni che utilizzano l'attributo "conto" sono: OP8, OP9, OP10, OP11.

OP8 senza "conto" ha la seguente tavola degli accessi:

Concetto	Costrutto	Accessi	Tipo
Comanda	E	1	L
ComandaCompletata	E	1	S

Costo totale: 150/giorno

OP9 con o senza "conto" non fa la differenza sulla tavola degli accessi.

OP10 senza "conto" ha la seguente tavola degli accessi:

Concetto	Costrutto	Accessi	Tipo
ComandaCompletata	E	50	L
Ordinazione	R	200	L
IstanzaProdotto	E	200	L

Costo totale: 450/giorno

OP11 senza "conto" ha la seguente tavola degli accessi:

Concetto	Costrutto	Accessi	Tipo
ComandaCompletata	E	1.521	L
Ordinazione	R	6.084	L
IstanzaProdotto	E	6.084	L

Costo totale: 13.689/mese

Confrontando il costo delle operazioni senza "conto" con quelle in cui "conto" è presente si conclude che la presenza di "conto" fa risparmiare 12.168 accessi al mese a fronte di un'occupazione di solamente 73.000 byte all'anno; quindi, ho deciso di mantenere l'attributo "conto".

Eliminazione delle generalizzazioni

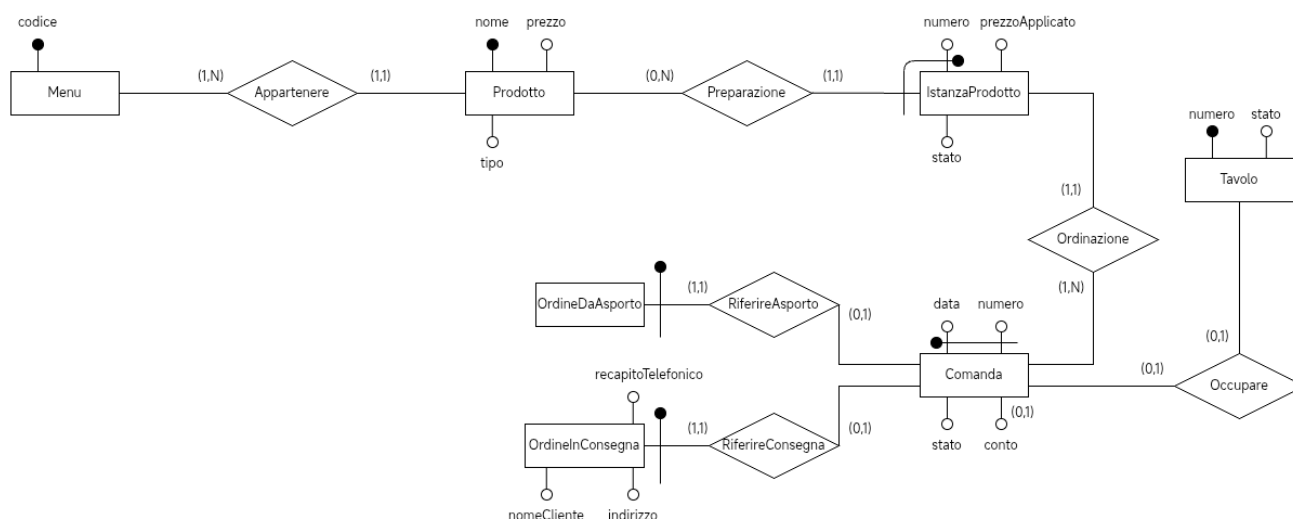
La generalizzazione totale di Prodotto, per mancanza di attributi aggiuntivi, viene eliminata accorpando le entità figlie nel genitore. Si è introdotto l'attributo discriminante "tipo" per indicare il tipo dell'occorrenza, che assume i valori "pizza" o "bevanda".

La generalizzazione totale di IstanzaProdotto, per mancanza di attributi aggiuntivi, viene eliminata accorpando le entità figlie nel genitore. Si è introdotto l'attributo discriminante "stato" per indicare il tipo dell'occorrenza, che assume i valori "pronta" o "non pronta".

La generalizzazione totale di Tavolo, per mancanza di attributi aggiuntivi, viene eliminata accorpando le entità figlie nel genitore. Si è introdotto l'attributo discriminante "stato" per indicare il tipo dell'occorrenza, che assume i valori "libero" o "occupato".

La generalizzazione totale di Ordine, per la presenza di attributi aggiuntivi e di operazioni che si riferiscono solo a ordini da asporto oppure in consegna, viene eliminata accorpando l'entità genitore nelle figlie. L'associazione con Comanda viene quindi duplicata.

La generalizzazione parziale di Comanda, siccome l'attributo aggiuntivo "conto" ha valore nullo temporaneamente dato che a fine giornata tutte le comande vengono completate, viene eliminata accorpando l'entità figlia nel genitore. Si è introdotto l'attributo discriminante "stato" per indicare il tipo dell'occorrenza, che assume i valori "completata" o "aperta". L'attributo "conto" diventa opzionale.

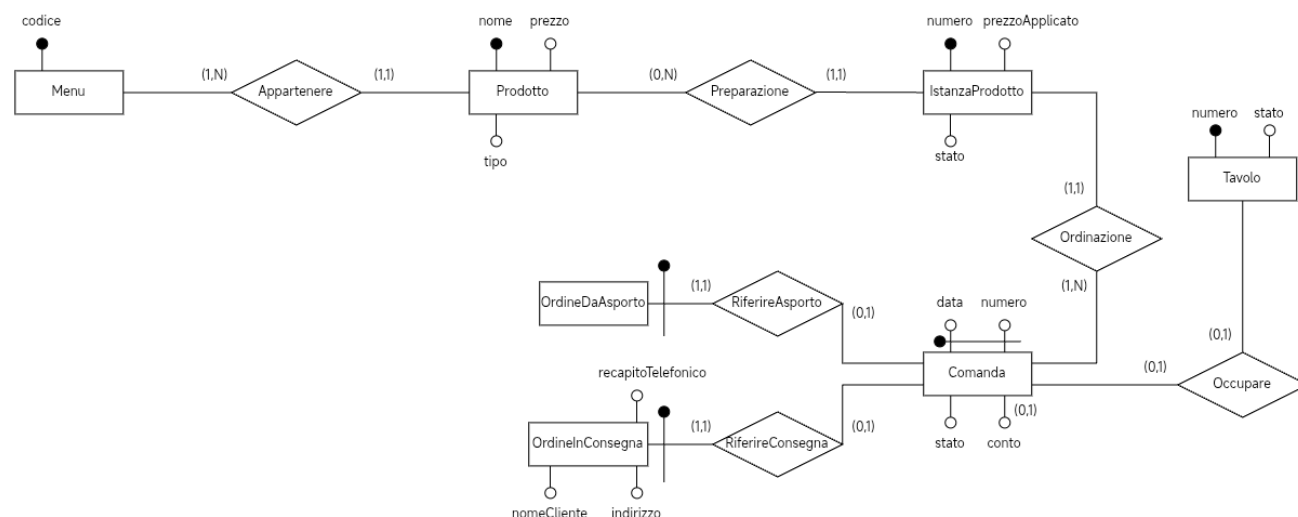


Scelta degli identificatori primari

Gli identificatori primari sono quelli visibili sullo schema ER ristrutturato, tutti gli identificatori primari sono stati scelti assicurando il vincolo di non nullabilità.

Trasformazione di attributi e identificatori

Per facilitare la traduzione nello schema relazionale, l'identificatore esterno di IstanzaProdotto viene eliminato e sostituito da un identificatore interno artificiale. IstanzaProdotto diventa quindi un'entità forte.



Traduzione di entità e associazioni

Menu(Codice)

Prodotto(Nome, Tipo, Prezzo, CodiceMenu)

Comanda(Data, Numero, Stato, Conto*)

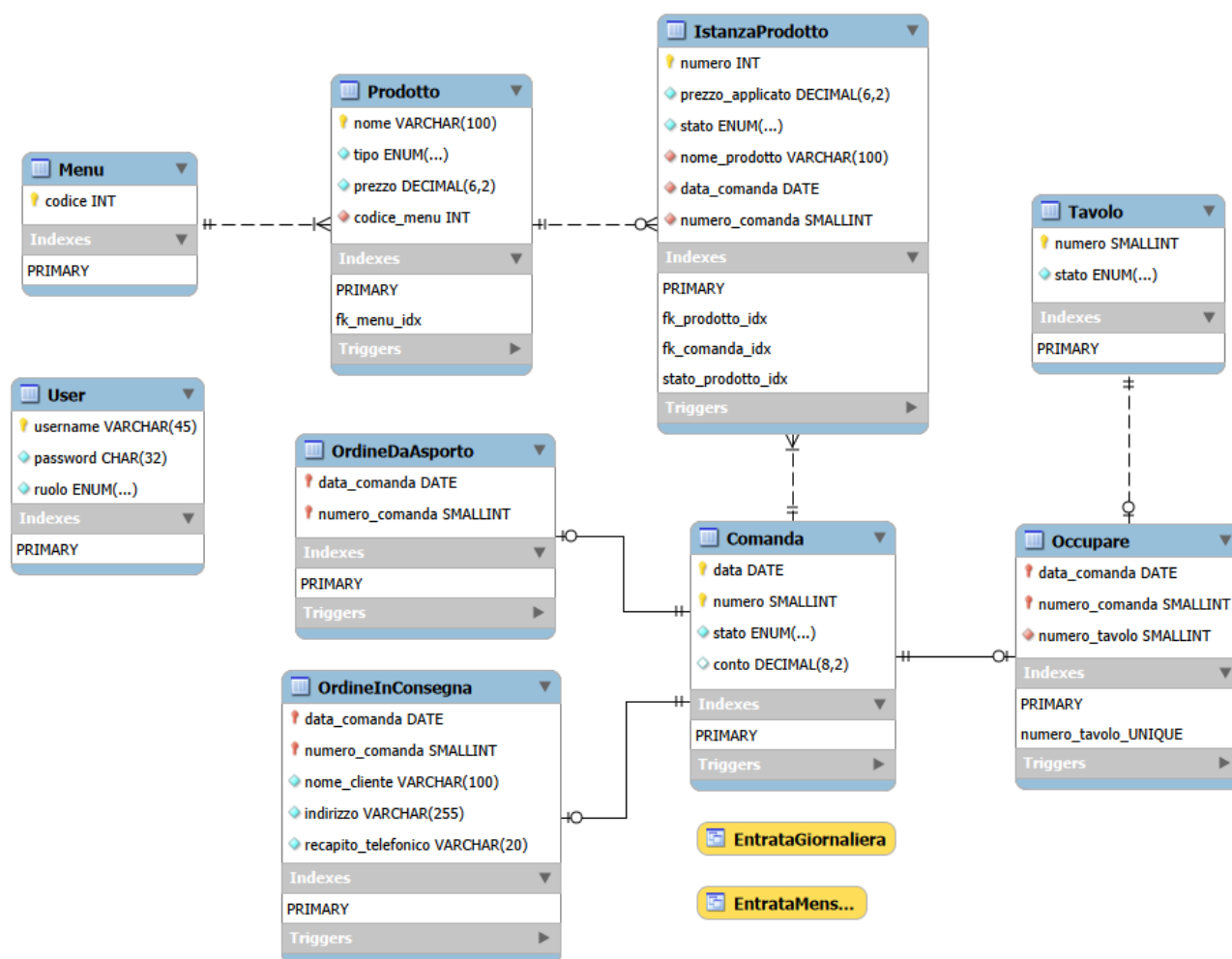
Tavolo(Numero, Stato)

IstanzaProdotto(Numero, PrezzoApplicato, Stato, NomeProdotto, DataComanda, NumeroComanda)

OrdineDaAsporto(DataComanda, NumeroComanda)

OrdineInConsegna(DataComanda, NumeroComanda, NomeCliente, Indirizzo, RecapitoTelefonico)

Occupare(DataComanda, NumeroComanda, NumeroTavolo)



Normalizzazione del modello relazionale

Il modello relazionale risulta in forma normale di Boyce-Codd (BCNF) poiché, in ogni relazione, per ogni dipendenza funzionale non banale $X \rightarrow A$, X è una superchiave della relazione.

5. Progettazione fisica

Utenti e privilegi

L'assegnazione dei privilegi segue il principio del privilegio minimo (Principle of Least Privilege).

Si prevedono quattro ruoli:

1. Manager
 - a. Grant di esecuzione sulle procedure OP2, OP8, OP9, OP10, OP11, OP12, OP13, OP14, OP15.
2. Cameriere
 - a. Grant di esecuzione sulle procedure OP1, OP2, OP7.
3. Pizzaiolo
 - a. Grant di esecuzione sulle procedure OP3, OP5.
4. Barman
 - a. Grant di esecuzione sulle procedure OP4, OP6.

Per identificare gli utenti si introduce una tabella User per mantenerne le credenziali.

Strutture di memorizzazione

Tabella User		
Colonna	Tipo di dato	Attributi ²
username	VARCHAR(45)	PK, NN
password	CHAR(32)	NN
ruolo	ENUM('manager', 'cameriere', 'pizzaiolo', 'barman')	NN

Tabella Menu		
Colonna	Tipo di dato	Attributi ²
codice	INT	PK, NN, UN

Tabella Prodotto		
Colonna	Tipo di dato	Attributi ²
nome	VARCHAR(100)	PK, NN
tipo	ENUM('pizza', 'bevanda')	NN
prezzo	DECIMAL(6,2)	NN
codice_menu	INT	NN, UN

Tabella IstanzaProdotto		
Colonna	Tipo di dato	Attributi ²
numero	INT	PK, NN, UN, AI
prezzo applicato	DECIMAL(6,2)	NN

stato	ENUM('non pronta', 'pronta')	NN
nome prodotto	VARCHAR(100)	NN
data comanda	DATE	NN
numero comanda	SMALLINT	NN, UN

Tabella Comanda		
Colonna	Tipo di dato	Attributi ²
data	DATE	PK, NN
numero	SMALLINT	PK, NN, UN
stato	ENUM('aperta', 'completata')	NN
conto	DECIMAL(8,2)	

Tabella Occupare		
Colonna	Tipo di dato	Attributi ²
data comanda	DATE	PK, NN
numero comanda	SMALLINT	PK, NN, UN
numero tavolo	SMALLINT	NN, UN, UQ

Tabella Tavolo		
Colonna	Tipo di dato	Attributi ²
numero	SMALLINT	PK, NN, UN
stato	ENUM('libero', 'occupato')	NN

Tabella OrdineDaAsporto		
Colonna	Tipo di dato	Attributi ²
data comanda	DATE	PK, NN
numero comanda	SMALLINT	PK, NN, UN

Tabella OrdineInConsegna		
Colonna	Tipo di dato	Attributi ²
data comanda	DATE	PK, NN
numero comanda	SMALLINT	PK, NN, UN
nome cliente	VARCHAR(100)	NN
indirizzo	VARCHAR(255)	NN
recapito telefonico	VARCHAR(20)	NN

Indici

Tabella Occupare	
Indice numero_tavolo_UNIQUE	Tipo ³ :
numero tavolo	UQ

² PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

³ IDX = index, UQ = unique, FT = full text, PR = primary.

Impedisce che un tavolo occupato abbia contemporaneamente più comande.

Tabella IstanzaProdotto	
Indice stato prodotto idx	Tipo ³ :
stato	IDX
nome prodotto	IDX

Facilita la consultazione dei prodotti non pronti/pronti da parte di pizzaiolo, barman e cameriere (OP3, OP4, OP7).

Trigger

Tabella Prodotto: BEFORE UPDATE

Il seguente trigger implementa la terza regola di vincolo.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Pizzeria`.`trg_prodotto_bu_prezzo` BEFORE
UPDATE ON `Prodotto` FOR EACH ROW
BEGIN
  IF NEW.prezzo <= 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Il prezzo del prodotto deve essere maggiore di zero';
  END IF;
END
```

Tabella Prodotto: BEFORE INSERT

Il seguente trigger implementa la terza regola di vincolo.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Pizzeria`.`trg_prodotto_bi_prezzo` BEFORE
INSERT ON `Prodotto` FOR EACH ROW
BEGIN
  IF NEW.prezzo <= 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Il prezzo del prodotto deve essere maggiore di zero';
  END IF;
END
```

Tabella IstanzaProdotto: BEFORE INSERT

Il seguente trigger implementa la quarta regola di derivazione.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Pizzeria`.`trg_istanza_bi_prezzo` BEFORE
INSERT ON `IstanzaProdotto` FOR EACH ROW
BEGIN
    SET NEW.prezzo_applicato = (
        SELECT prezzo
        FROM Prodotto
        WHERE nome = NEW.nome_prodotto
    );
END
```

Tabella Comanda: BEFORE UPDATE

Il seguente trigger implementa la prima regola di derivazione e la quarta regola di vincolo.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Pizzeria`.`trg_comanda_bu_conto` BEFORE
UPDATE ON `Comanda` FOR EACH ROW
BEGIN
    DECLARE v_conto DECIMAL(8,2);

    IF OLD.stato = 'aperta' AND NEW.stato = 'completata' THEN
        IF NOT EXISTS (
            SELECT 1
            FROM IstanzaProdotto
            WHERE data_comanda = OLD.data AND numero_comanda = OLD.numero
        ) THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Non è possibile completare una comanda senza prodotti';
        END IF;

        IF EXISTS (
            SELECT 1
            FROM IstanzaProdotto
            WHERE data_comanda = OLD.data
```

```
        AND numero_comanda = OLD.numero
        AND stato = 'non pronta'
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La comanda contiene prodotti non ancora pronti';
    END IF;

    SELECT SUM(prezzo_applicato)
    INTO v_conto
    FROM IstanzaProdotto
    WHERE data_comanda = OLD.data AND numero_comanda = OLD.numero;

    SET NEW.conto = v_conto;
END IF;
END
```

Tabella Comanda: BEFORE INSERT

Il seguente trigger implementa la seconda regola di vincolo.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Pizzeria`.`trg_comanda_bi_numero`
BEFORE INSERT ON `Comanda` FOR EACH ROW
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM Comanda
        WHERE data = NEW.data
    ) AND NEW.numero <> 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La prima comanda della giornata deve avere numero zero';
    END IF;
END
```

Tabella OrdineDaAsporto: BEFORE INSERT

Il seguente trigger implementa la prima regola di vincolo.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Pizzeria`.`trg_asporto_bi_tavolo` BEFORE
INSERT ON `OrdineDaAsporto` FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1
        FROM Occupare
        WHERE data_comanda = NEW.data_comanda AND numero_comanda =
NEW.numero_comanda
    )
    OR EXISTS (
        SELECT 1
        FROM OrdineInConsegna
        WHERE data_comanda = NEW.data_comanda AND numero_comanda =
NEW.numero_comanda
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La comanda è già associata a un tavolo o a un ordine in consegna';
    END IF;
END
```

Tabella OrdineInConsegna: BEFORE INSERT

Il seguente trigger implementa la prima regola di vincolo.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Pizzeria`.`trg_consegna_bi_tavolo` BEFORE
INSERT ON `OrdineInConsegna` FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1
        FROM Occupare
        WHERE data_comanda = NEW.data_comanda AND numero_comanda =
NEW.numero_comanda
    )
```

```
OR EXISTS (  
    SELECT 1  
    FROM OrdineDaAsporto  
    WHERE data_comanda = NEW.data_comanda AND numero_comanda =  
NEW.numero_comanda  
) THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'La comanda è già associata a un tavolo o a un ordine da asporto';  
END IF;  
END
```

Tabella Occupare: BEFORE INSERT

Il seguente trigger implementa la prima regola di vincolo.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Pizzeria`.`trg_occupare_bi_ordine` BEFORE  
INSERT ON `Occupare` FOR EACH ROW  
BEGIN  
    IF EXISTS (  
        SELECT 1  
        FROM OrdineDaAsporto  
        WHERE data_comanda = NEW.data_comanda AND numero_comanda =  
NEW.numero_comanda  
    ) OR EXISTS (  
        SELECT 1  
        FROM OrdineInConsegna  
        WHERE data_comanda = NEW.data_comanda AND numero_comanda =  
NEW.numero_comanda  
    ) THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'La comanda è già associata a un ordine da asporto o a un ordine in  
consegna';  
    END IF;  
END
```

Tabella Occupare: AFTER INSERT

Il seguente trigger segna il tavolo come “occupato” quando viene creata una comanda associata ad esso.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Pizzeria`.`trg_occupare_ai_tavolo` AFTER
INSERT ON `Occupare` FOR EACH ROW
BEGIN
    UPDATE Tavolo
    SET stato = 'occupato'
    WHERE numero = NEW.numero_tavolo;
END
```

Tabella Occupare: BEFORE DELETE

Il seguente trigger impedisce di liberare il tavolo se la comanda associata ad esso non è stata ancora completata.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Pizzeria`.`trg_occupare_bd_comanda`
BEFORE DELETE ON `Occupare` FOR EACH ROW
BEGIN
    IF (
        SELECT stato
        FROM Comanda
        WHERE data = OLD.data_comanda AND numero = OLD.numero_comanda
    ) <> 'completata' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Il tavolo può essere liberato solo dopo il completamento della
comanda';
    END IF;
END
```

Tabella Occupare: AFTER DELETE

Il seguente trigger segna come “libero” il tavolo a cui era associata una comanda che è stata appena completata.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Pizzeria`.`trg_occupare_ad_tavolo` AFTER
DELETE ON `Occupare` FOR EACH ROW
BEGIN
    UPDATE Tavolo
    SET stato = 'libero'
    WHERE numero = OLD.numero_tavolo;
END
```

Eventi

Non sono stati utilizzati eventi.

Viste**Vista per le entrate giornaliere**

La vista EntrataGiornaliera supporta l'operazione OP10 e restituisce una riga per ogni giorno in cui è stata completata almeno una comanda. La clausola WHERE seleziona solamente le comande completate, mentre GROUP BY le raggruppa in base alla giornata. La funzione SUM(conto) calcola la somma dei conti di ciascun gruppo. La vista implementa quindi la seconda regola di derivazione.

```
CREATE VIEW EntrataGiornaliera AS
SELECT data, SUM(conto) AS entrata_giornaliera
FROM Comanda
WHERE stato = 'completata'
GROUP BY data;
```

Vista per le entrate mensili

La vista EntrataMensile supporta l'operazione OP11 e restituisce una riga per ogni coppia anno-mese in cui è stata completata almeno una comanda. La clausola WHERE considera solamente le comande completate, mentre GROUP BY le raggruppa in base all'anno e al mese. La funzione SUM(conto) calcola l'entrata complessiva di ciascun gruppo. La vista implementa quindi la terza regola di derivazione.

```
CREATE VIEW EntrataMensile AS
SELECT YEAR(data) AS anno, MONTH(data) AS mese, SUM(conto) AS entrata_mensile
FROM Comanda
WHERE stato = 'completata'
GROUP BY YEAR(data), MONTH(data);
```

Stored Procedures e transazioni

Operazione Login:

```
CREATE PROCEDURE `login` (
  IN p_username VARCHAR(45),
  IN p_password VARCHAR(100),
  OUT p_ruolo INT
)
BEGIN
  DECLARE v_ruolo ENUM(
    'manager',
    'cameriere',
    'pizzaiolo',
    'barman'
  );

  SELECT ruolo
  INTO v_ruolo
  FROM `User`
  WHERE username = p_username AND password = MD5(p_password);

  IF v_ruolo = 'manager' THEN
    SET p_ruolo = 1;
  ELSEIF v_ruolo = 'cameriere' THEN
    SET p_ruolo = 2;
  ELSEIF v_ruolo = 'pizzaiolo' THEN
    SET p_ruolo = 3;
  ELSEIF v_ruolo = 'barman' THEN
```

```
    SET p_ruolo = 4;
ELSE
    SET p_ruolo = 5;
END IF;
END
```

Operazione Crea User:

```
CREATE PROCEDURE `crea_user` (
    IN p_username VARCHAR(45),
    IN p_password VARCHAR(100),
    IN p_ruolo VARCHAR(20)
)
BEGIN
    IF TRIM(p_username) = "" OR p_password = "" THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Username e password non possono essere vuoti';
    END IF;

    INSERT INTO `User` (username, password, ruolo)
    VALUES (p_username, MD5(p_password), p_ruolo);
END
```

Operazione OP1:

Il livello di isolamento SERIALIZABLE evita le phantom reads su Comanda che porterebbero ad un calcolo sbagliato del numero della comanda.

```
CREATE PROCEDURE `apri_comanda_tavolo` (
    IN p_numero_tavolo SMALLINT UNSIGNED,
    OUT p_data_comanda DATE,
    OUT p_numero_comanda SMALLINT UNSIGNED
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
```

```
ROLLBACK;
RESIGNAL;
END;

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;

SET p_data_comanda = CURRENT_DATE();

SELECT COALESCE(MAX(numero) + 1, 0)
INTO p_numero_comanda
FROM Comanda
WHERE data = p_data_comanda;

INSERT INTO Comanda (data, numero, stato, conto)
VALUES (p_data_comanda, p_numero_comanda, 'aperta', NULL);

INSERT INTO Occupare (data_comanda, numero_comanda, numero_tavolo)
VALUES (p_data_comanda, p_numero_comanda, p_numero_tavolo);

COMMIT;
END
```

Operazione OP2:

Il livello di isolamento REPEATABLE READ impedisce che la comanda letta, a cui sta venendo aggiunta l'ordinazione, venga completata prima del termine della transazione.

```
CREATE PROCEDURE `aggiungi_ordinazione` (
  IN p_data_comanda DATE,
  IN p_numero_comanda SMALLINT UNSIGNED,
  IN p_nome_prodotto VARCHAR(100)
)
BEGIN
  DECLARE v_stato_comanda VARCHAR(15);
```

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    RESIGNAL;
END;

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;

SET v_stato_comanda = (
    SELECT stato
    FROM Comanda
    WHERE data = p_data_comanda AND numero = p_numero_comanda
);

IF v_stato_comanda = 'completata' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Non è possibile aggiungere prodotti a una comanda completata';
END IF;

INSERT INTO IstanzaProdotto (nome_prodotto, data_comanda, numero_comanda)
VALUES (p_nome_prodotto, p_data_comanda, p_numero_comanda);

COMMIT;
END
```

Operazione OP3:

Il livello di isolamento READ COMMITTED evita le dirty reads su IstanzaProdotto.

```
CREATE PROCEDURE `pizze_da_preparare` ()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
  SET TRANSACTION READ ONLY;
  START TRANSACTION;

  SELECT ip.numero, ip.nome_prodotto
  FROM IstanzaProdotto AS ip
  JOIN Prodotto AS p ON p.nome = ip.nome_prodotto
  WHERE ip.stato = 'non pronta' AND p.tipo = 'pizza'
  ORDER BY ip.numero;

  COMMIT;
END
```

Operazione OP4:

Il livello di isolamento READ COMMITTED evita le dirty reads su IstanzaProdotto.

```
CREATE PROCEDURE `bevande_da_preparare` ()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;
END;
```

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
SET TRANSACTION READ ONLY;  
START TRANSACTION;
```

```
SELECT ip.numero, ip.nome_prodotto  
FROM IstanzaProdotto AS ip  
JOIN Prodotto AS p ON p.nome = ip.nome_prodotto  
WHERE ip.stato = 'non pronta' AND p.tipo = 'bevanda'  
ORDER BY ip.numero;
```

```
COMMIT;  
END
```

Operazione OP5:

```
CREATE PROCEDURE `segna_pizza_pronta` (  
  IN p_numero_istanza INT UNSIGNED  
)  
BEGIN  
  UPDATE IstanzaProdotto AS ip  
  JOIN Prodotto AS p ON p.nome = ip.nome_prodotto  
  SET ip.stato = 'pronta'  
  WHERE ip.numero = p_numero_istanza AND ip.stato = 'non pronta' AND p.tipo = 'pizza';  
END
```

Operazione OP6:

```
CREATE PROCEDURE `segna_bevanda_pronta` (  
  IN p_numero_istanza INT UNSIGNED  
)  
BEGIN  
  UPDATE IstanzaProdotto AS ip  
  JOIN Prodotto AS p ON p.nome = ip.nome_prodotto  
  SET ip.stato = 'pronta'  
  WHERE ip.numero = p_numero_istanza
```

```
AND ip.stato = 'non pronta'  
AND p.tipo = 'bevanda';  
END
```

Operazione OP7:

Il livello di isolamento READ COMMITTED evita le dirty reads su IstanzaProdotto.

```
CREATE PROCEDURE `prodotti_pronti_tavoli` ()  
BEGIN  
  DECLARE EXIT HANDLER FOR SQLEXCEPTION  
  BEGIN  
    ROLLBACK;  
    RESIGNAL;  
  END;  
  
  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
  SET TRANSACTION READ ONLY;  
  START TRANSACTION;  
  
  SELECT ip.numero, ip.nome_prodotto, o.numero_tavolo  
  FROM IstanzaProdotto AS ip  
  JOIN Occupare AS o  
    ON o.data_comanda = ip.data_comanda  
    AND o.numero_comanda = ip.numero_comanda  
  JOIN Comanda AS c  
    ON c.data = ip.data_comanda  
    AND c.numero = ip.numero_comanda  
  WHERE ip.stato = 'pronta'  
    AND c.stato = 'aperta'  
  ORDER BY o.numero_tavolo, ip.numero;  
  
  COMMIT;  
END
```

Operazione OP8:

Il livello di isolamento SERIALIZABLE evita le phantom reads su IstanzaProdotto, siccome il trigger BEFORE UPDATE di Comanda controlla che tutte le istanze della comanda siano pronte prima di poterla completare; quindi, evita l'inserimento in concorrenza di una nuova istanza di prodotto non pronta.

```
CREATE PROCEDURE `completa_comanda` (  
    IN p_data_comanda DATE,  
    IN p_numero_comanda SMALLINT UNSIGNED  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
    START TRANSACTION;  
  
    UPDATE Comanda  
    SET stato = 'completata'  
    WHERE data = p_data_comanda  
        AND numero = p_numero_comanda  
        AND stato = 'aperta';  
  
    COMMIT;  
END
```

Operazione OP9:

Il livello di isolamento REPEATABLE READ evita che le due SELECT leggano due stati diversi della stessa comanda.

```
CREATE PROCEDURE `genera_scontrino` (  
    IN p_data_comanda DATE,  
    IN p_numero_comanda SMALLINT UNSIGNED  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
    SET TRANSACTION READ ONLY;  
    START TRANSACTION;  
  
    SELECT ip.nome_prodotto,  
        COUNT(*) AS quantita,  
        ip.prezzo_applicato AS prezzo_unitario,  
        SUM(ip.prezzo_applicato) AS subtotale  
    FROM IstanzaProdotto AS ip  
    JOIN Comanda AS c  
        ON c.data = ip.data_comanda  
        AND c.numero = ip.numero_comanda  
    WHERE ip.data_comanda = p_data_comanda  
        AND ip.numero_comanda = p_numero_comanda  
        AND c.stato = 'completata'  
    GROUP BY ip.nome_prodotto, ip.prezzo_applicato  
    ORDER BY ip.nome_prodotto;  
  
    SELECT conto AS totale
```

```
FROM Comanda
WHERE data = p_data_comanda
      AND numero = p_numero_comanda
      AND stato = 'completata';

COMMIT;
END
```

Operazione OP10:

Il livello di isolamento READ COMMITTED evita le dirty reads su Comanda.

```
CREATE PROCEDURE `entrata_giornaliera` (
    IN p_data DATE
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
    SET TRANSACTION READ ONLY;
    START TRANSACTION;

    SELECT data, entrata_giornaliera
    FROM EntrataGiornaliera
    WHERE data = p_data;

    COMMIT;
END
```

Operazione OP11:

Il livello di isolamento READ COMMITTED evita le dirty reads su Comanda.

```
CREATE PROCEDURE `entrata_mensile` (  
    IN p_anno_mese CHAR(7)  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    SET TRANSACTION READ ONLY;  
    START TRANSACTION;  
  
    IF p_anno_mese IS NULL OR p_anno_mese NOT REGEXP '^[1-9][0-9]{3}-(0[1-9]|1[0-2])$'  
    THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Il periodo deve essere espresso nel formato YYYY-MM';  
    END IF;  
  
    SELECT anno, mese, entrata_mensile  
    FROM EntrataMensile  
    WHERE anno = CAST(LEFT(p_anno_mese, 4) AS UNSIGNED)  
        AND mese = CAST(RIGHT(p_anno_mese, 2) AS UNSIGNED);  
  
    COMMIT;  
END
```

Operazione OP12:

```
CREATE PROCEDURE `libera_tavolo` (  
  IN p_numero_tavolo SMALLINT UNSIGNED  
)  
BEGIN  
  DELETE FROM Occupare  
  WHERE numero_tavolo = p_numero_tavolo;  
END
```

Operazione OP13:

Il livello di isolamento SERIALIZABLE evita le phantom reads su Comanda che porterebbero ad un calcolo sbagliato del numero della comanda.

```
CREATE PROCEDURE `registra_asporto` (  
  OUT p_data_comanda DATE,  
  OUT p_numero_comanda SMALLINT UNSIGNED  
)  
BEGIN  
  DECLARE EXIT HANDLER FOR SQLEXCEPTION  
  BEGIN  
    ROLLBACK;  
    RESIGNAL;  
  END;  
  
  SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
  START TRANSACTION;  
  
  SET p_data_comanda = CURRENT_DATE();  
  
  SELECT COALESCE(MAX(numero) + 1, 0)  
  INTO p_numero_comanda  
  FROM Comanda  
  WHERE data = p_data_comanda;
```

```
INSERT INTO Comanda (data, numero, stato, conto)
VALUES (p_data_comanda, p_numero_comanda, 'aperta', NULL);

INSERT INTO OrdineDaAsporto (data_comanda, numero_comanda)
VALUES (p_data_comanda, p_numero_comanda);

COMMIT;

END
```

Operazione OP14:

Il livello di isolamento SERIALIZABLE evita le phantom reads su Comanda che porterebbero ad un calcolo sbagliato del numero della comanda.

```
CREATE PROCEDURE `registra_consegna` (
  IN p_nome_cliente VARCHAR(100),
  IN p_indirizzo VARCHAR(255),
  IN p_recapito_telefonico VARCHAR(20),
  OUT p_data_comanda DATE,
  OUT p_numero_comanda SMALLINT UNSIGNED
)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
  START TRANSACTION;

  SET p_data_comanda = CURRENT_DATE();

  SELECT COALESCE(MAX(numero) + 1, 0)
  INTO p_numero_comanda
```

```
FROM Comanda
WHERE data = p_data_comanda;

INSERT INTO Comanda (data, numero, stato, conto)
VALUES (p_data_comanda, p_numero_comanda, 'aperta', NULL);

INSERT INTO OrdineInConsegna (data_comanda, numero_comanda, nome_cliente, indirizzo,
recapito_telefonico)
VALUES (p_data_comanda, p_numero_comanda, p_nome_cliente, p_indirizzo,
p_recapito_telefonico);

COMMIT;
END
```

Operazione OP15:

```
CREATE PROCEDURE `modifica_prezzo` (
  IN p_nome_prodotto VARCHAR(100),
  IN p_nuovo_prezzo DECIMAL(6,2)
)
BEGIN
  UPDATE Prodotto
  SET prezzo = p_nuovo_prezzo
  WHERE nome = p_nome_prodotto;
END
```